versor-java 2.0

架构、稠密/稀疏乘积与端到端性能

几何代数核心 · 纯数学，无 Minecraft 依赖

2026-10-21

1 摘要

- 稠密几何积仍是唯一 4^n 热点：n=10 为 0.86 ms，每对约 0.82 ns；输出驱动与 FMA 变体实测更慢（n=10 为 1.47–1.50 ms），保留现有循环。
- 核心 `gp` 增加自适应稀疏支撑：PGA motor 乘点 83.3 ns；Cl(10) 仅 10 个非零系数时 2.9 μs，而稠密 0.86 ms。
- 外积/缩并改为子掩码枚举（$4^n \to 3^n$）：n=10 外积 79.6 μs，原 633 μs（8.1×）。
- 稀疏 COO：n=10、k=10 的 `gp` 1.16 μs，约 742× 于稠密；n=14、k=512 的 `gp`/`sandwich` 为 14/403 ms。
- 批量输出复用把 1M 点从 51.2 降到 20.0 ns/点（分配 128 → 0 B/点）；4 线程达到带宽平台。
- 端到端 1M 点：GA 36.7、批量 20.1、缓存 Matrix4f 1.60、adapter 349 ns/点。
- 多核 20 线程：gp 8.48×、op 7.88×、批量 2.47×、标量 3.86×。
- 断言：243 项通过；稠密/稀疏逐位一致，子掩码与暴力参考一致；`inverseGeneral` 采用均衡化、迭代精化与后向误差检查，近奇异结果默认拒绝。
- SIMD 原型（jdk.incubator.vector）：E3 批量 0.8–1.0×、PGA3 批量 1.3–1.9×；仅保留为可选内核，生产路径仍用标量。

2 架构与不变量

库保持单包、分层明确，不引入运行时框架：

层	类型	职责
代数	`Signature`	度量、位掩码 blade、grade、乘积/平方/反转符号
稠密	`Multivector`	按 mask 的系数数组；4^n 内核 + 稀疏支撑快速路径
稀疏	`SparseMultivector`	COO；哈希/稠密累加器按规模切换
批量	`MultivectorBatch`	SoA；输出复用、分块切片
矩阵	`MultivectorMatrix`	多向量矩阵乘/转置/Gauss–Jordan 求逆
模型	`E3`/`P3`/`Conformal`	向量、转子、motor、点线面与快捷公式
运动	`Motion`/`Field`/`Calculus`/`Manifold`	twist、场微分、参数化流形

不变量：所有操作数共享同一 `Signature` 实例；blade 按 mask 排序；零度量 生成元平方为零；$n \le 10$ 惰性构建符号表，$n = 11$–14 按需计算符号。 2.0 将反转/共轭/对合符号与 blade 平方符号收敛到 `Signature`， 消除了稀疏、稠密、批量三处的重复实现。

3　方法

环境：12th Gen Intel(R) Core(TM) i7-12700H；openjdk version ”25.0.3” 2026-04-21；JOML 1.10.9。每个测量点自动校准到约 40 ms，预热 3 轮、取 5 轮最小值；JMH 使用 1.37， 3 轮预热 + 4 轮测量。除多核一节外均为单线程、非隔离环境，绝对纳秒有 ±10% 抖动。完整矩阵由 `bench-all.sh` 一次运行，原始日志与 CSV 在 `report/data`。

4　稠密乘积内核

运算	n=3	n=4	n=5	n=6	n=7	n=8	n=9	n=10
gp	79.7	280.5	906.8	3,865	14,257	54,579	216,154	861,543
op	38.9	102.8	293.5	851.1	2,635	8,049	24,006	79,582
ip	38.2	101.4	289.4	851.2	2,568	7,655	22,977	80,175
rightContraction	41.6	105.9	303.6	874.3	2,577	7,873	22,919	78,760
commutator	170.7	560.0	1,854	7,777	28,133	108,819	436,911	1,711,302
reverse	10.6	16.1	28.5	54.9	108.8	248.2	585.4	1,201
dual	25.5	50.2	115.5	267.6	669.3	1,530	3,618	18,193
norm2	89.5	284.8	937.2	3,714	14,389	54,684	220,930	860,413
unit	93.8	283.7	963.8	3,544	14,333	52,510	221,778	871,941
inverse	62.7	104.0	157.3	293.4	595.3	1,269	2,693	5,808
sandwich	175.4	552.0	1,839	7,468	28,128	109,093	436,362	1,740,399
inverseGeneral	1,726	4,425	14,427	65,745	–	–	–	–
exp	1,047	1,529	2,134	3,685	–	–	–	–

几何积 `gp` 仍是唯一的 4^n 热点：n=10 约 0.86 ms， 每对 blade 约 0.82 ns。图 1 给出维度扫描与 4^n 参考。

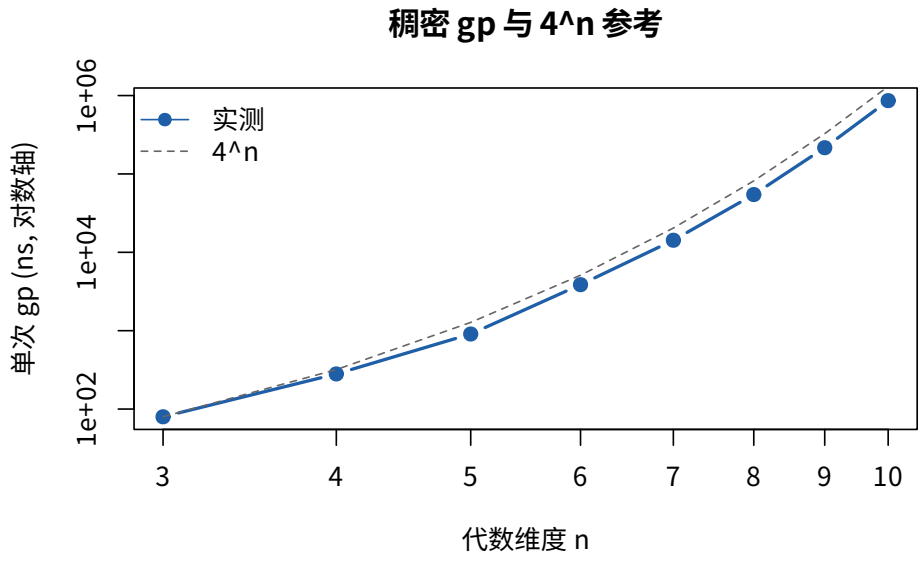


Figure 1: 稠密 `gp` 随维度增长；虚线为 4^n 参考。

压力场景与独立复核。 稠密 sandwich 与通用逆在高维迅速压过 `gp`； JMH 1.37 的单线程结果与手写 harness 一致（差异 < 10%）， 说明测得的 纳秒不是校准循环的假象。

n	稠密 sandwich	inverseGeneral	3x3 矩阵逆
4	563.8	2,971	13,731
5	1,861	11,935	13,555
6	7,685	55,193	13,211
7	27,497	280,897	–
8	109,780	1,786,731	–

n	gp	sandwich	inverse
3	81.3	174.9	66.8
4	259.3	531.3	95.0
6	3,792	7,843	274.8
8	58,785	125,683	1,107
10	894,803	1,758,491	5,100

快速路径	1t	4t
`e3Rotate`	13.8	14.2
`p3Transform`	31.4	37.9
`loopTransform`	29.5	38.6
`batchTransform`	15.9	22.0
`inverseGeneral`	1,566	1,918

路径	ns/op	对照
E3 rotate	10.9	通用 n=3 gp: 79.7
P3 motor×point	32.5	通用 n=4 gp: 280.5

稀疏支撑。`Multivector.gp` 现在先统计两侧非零数：当 $4\,n_a n_b < 4^n$ 时只遍历非零支撑对，否则走原稠密循环。PGA motor 乘点 83.3 ns、E3 rotor 乘向量 48.7 ns；Cl(10) 中 10 个非零系数时 2.9 μs，而稠密为 0.86 ms。

Cl(10) 非零数 k	gp ns
2	2,831
4	2,781
10	2,896
32	3,855
128	16,733
512	145,767
1024	397,374

子掩码枚举。外积、左/右缩并的有效配对只有 3^n 个。旧实现对每个 a 扫描全部 b 再判断；现在按兼容性枚举子掩码，复杂度降到 3^n。n=10 时外积 80 μs（原 633 μs，8.1×）。

n	运算	原 ns	新 ns	提升
3	`ip`	64.5	38.4	1.68×
3	`op`	63.3	38.5	1.64×
3	`rightContraction`	65.7	41.6	1.58×
6	`ip`	2,422	836.1	2.90×
6	`op`	2,468	845.8	2.92×
6	`rightContraction`	2,407	863.4	2.79×
8	`ip`	43,744	7,641	5.72×
8	`op`	44,760	8,016	5.58×
8	`rightContraction`	43,730	7,704	5.68×
10	`ip`	639,685	77,206	8.29×
10	`op`	632,995	78,024	8.11×
10	`rightContraction`	633,194	76,225	8.31×

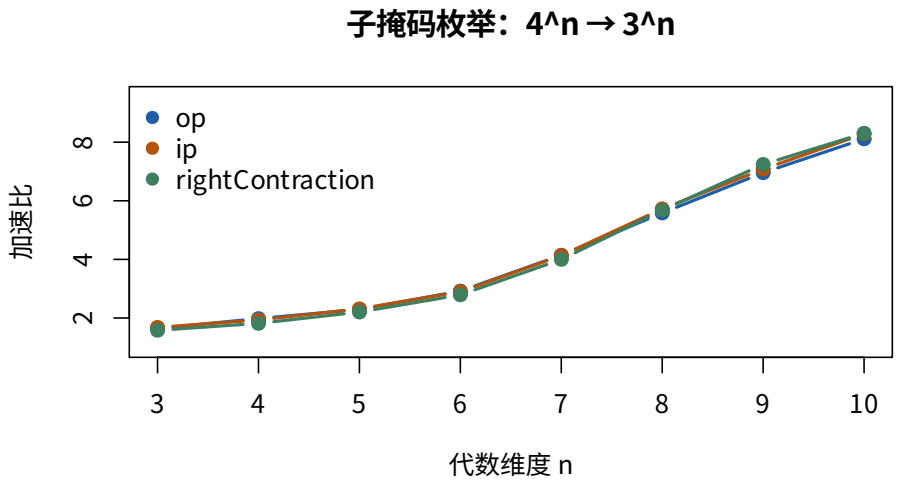


Figure 2: 子掩码枚举相对全扫描的加速比。

5 高维稀疏与符号表

n	k	gp ns	sandwich ns
10	8	616.8	3,130
10	32	5,738	54,690
10	512	247,655	738,485
12	8	1,194	10,553
12	32	18,505	1,081,651
12	512	11,302,185	95,343,026
14	8	1,500	10,469
14	32	27,678	1,649,695
14	512	13,719,937	403,249,548

COO 表示把乘积循环降到 k^2；低密度用开放寻址，高密度用稠密累加。 n=10、k=10（约 1%）时 `gp` 为 1.16 μs，相对稠密 0.86 ms 约 742×。n=14、k=512 的 `gp` 仍要约 14 ms，`sandwich` 因中间结果稠密化到约 403 ms。 $n \geq 11$ 不分配整张符号表，乘积符号按需计算。

n	k	hash	dense	auto	选择	判定
10	2	53.7	667.5	54.4	hash	ok
10	8	701.0	1,022	708.9	hash	ok
10	16	3,369	2,145	2,195	dense	ok
10	32	12,448	5,964	5,915	dense	ok
10	128	117,427	25,481	26,849	dense	ok
14	2	89.2	10,576	90.2	hash	ok
14	8	1,088	11,185	1,104	hash	ok
14	16	5,974	14,243	5,958	hash	ok
14	32	68,271	30,490	29,438	dense	ok
14	128	1,566,196	946,341	942,345	dense	ok
14	512	15,153,352	12,697,249	12,615,262	dense	ok

n	首次乘积 us	重建 us	表内存
8	1900.0	1957.0	0.2 MiB
9	9319.0	8545.0	1.0 MiB
10	36732.0	35225.0	4.0 MiB

结构化输入、混合密度与动态稀疏的结果见下表；sandwich 的退化来自中间 结果而非输入本身。

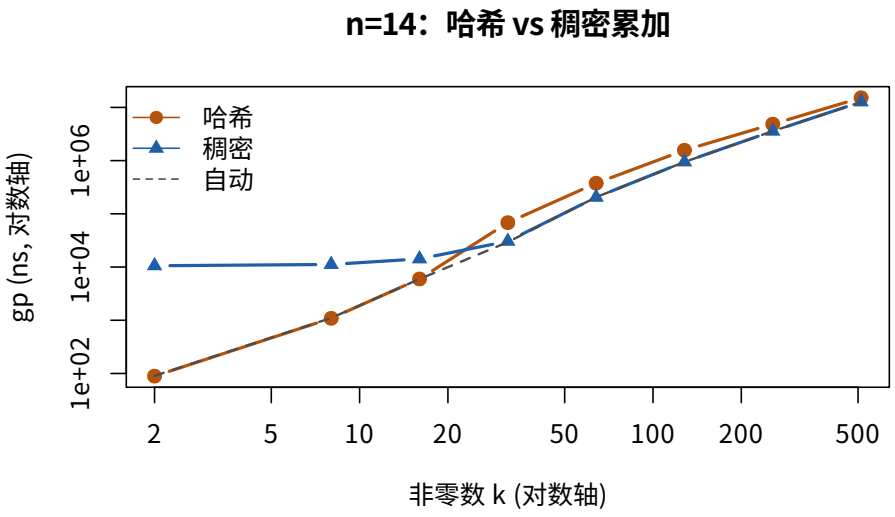


Figure 3: n=14 的哈希/稠密累加交叉；虚线为自动选择。

n	结构	k	gp ns	sandwich ns
10	uniform	8	23,062	3,462
10	versor	8	607.0	2,845
10	pgaPoint	8	687.2	2,991
10	uniform	32	5,780	54,372
10	versor	32	4,281	26,252
10	pgaPoint	32	4,332	25,917
14	uniform	8	1,420	10,736
14	versor	8	1,265	9,638
14	pgaPoint	8	1,182	9,169
14	uniform	32	28,298	1,632,712
14	versor	32	26,377	800,158
14	pgaPoint	32	24,728	533,564

6 批量、内存与多核

N	逐元素 ns/点	批量 ns/点	逐元素 B/点	批量 B/点
1	30.6	262.9	169	128
10	29.4	38.1	169	128
100	29.9	19.9	169	128
1000	27.1	18.1	169	128
10000	28.8	16.8	169	128
100000	32.2	17.8	169	128

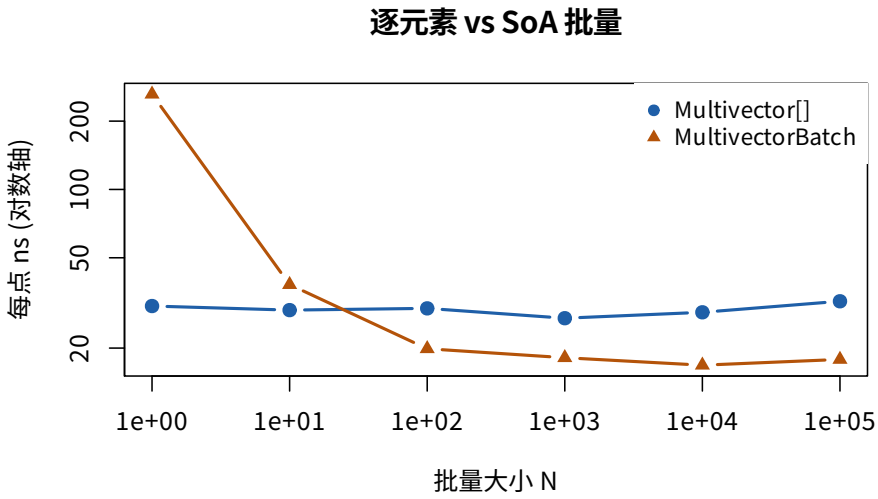


Figure 4: 逐元素与 SoA 批量的每点成本。

N	变体	ns/点	B/点	GB/s
100000	alloc	22.0	128	11.6
100000	reuse	11.7	0	21.8
100000	double	14.9	0	17.2
100000	aos	48.6	168	5.3
1000000	alloc	51.2	128	5.0
1000000	reuse	20.0	0	12.8
1000000	double	19.5	0	13.1
1000000	aos	58.7	168	4.4
10000000	alloc	59.1	128	4.3
10000000	reuse	18.3	0	14.0
10000000	double	19.0	0	13.5

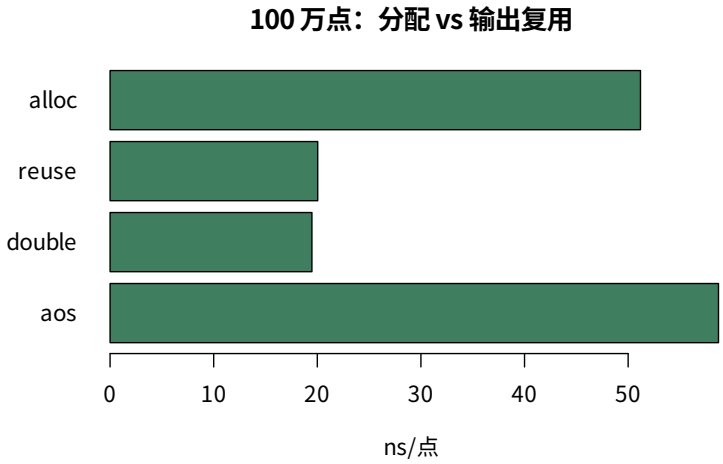


Figure 5: 100 万点下分配与输出复用的每点成本。

负载	线程	加速比	效率
gp_n10	1	1.00	100%
gp_n10	4	3.57	89%
gp_n10	8	5.20	65%
gp_n10	14	7.25	52%
gp_n10	20	8.48	42%
op_n10	1	1.00	100%
op_n10	4	3.75	94%
op_n10	8	4.72	59%
op_n10	14	7.01	50%
op_n10	20	7.88	39%
batch_sandwich	1	1.00	100%
batch_sandwich	4	2.29	57%
batch_sandwich	8	2.29	29%
batch_sandwich	14	2.13	15%
batch_sandwich	20	2.47	12%
scalar_sandwich	1	1.00	100%
scalar_sandwich	4	2.59	65%
scalar_sandwich	8	3.20	40%
scalar_sandwich	14	3.77	27%
scalar_sandwich	20	3.86	19%

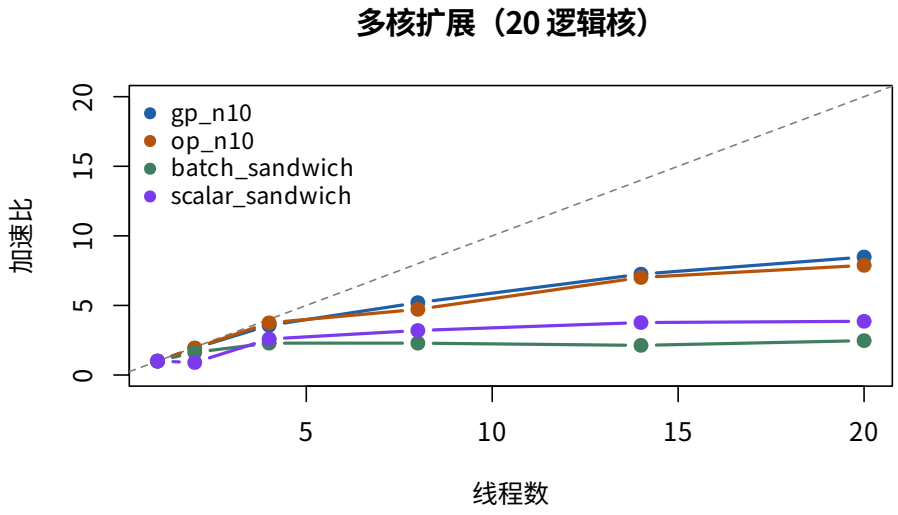


Figure 6: 多核加速比；虚线为线性扩展。

7 快速路径、端到端与机器人

路径	运算	ns/op
E3	rotate	10.9
JOML	rotateAlloc	2.9
P3	applyPoint	32.5
JOML	matrixTransformAlloc	2.3
E3	toQuaternion	144.6
JOML	toMatrix4f	333.4

JOML 列是 float、单次调用，JIT 可能对标量运算做常量折叠或向量化，不能直接 当作实际渲染成

本；下面 1M 点的端到端表才是可比口径。

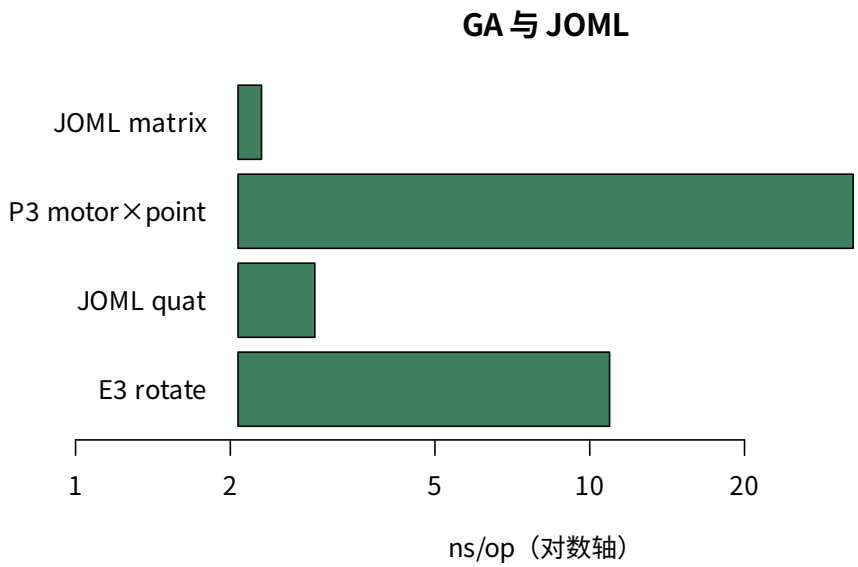


Figure 7: GA 与 JOML 的运算/转换成本。

路径	点数	ns/点	p99 帧 ms	分配 MB/帧
`ga`	1000000	36.7	40.7	168
`batch`	1000000	20.1	22.7	128
`matrix`	1000000	1.6	2.3	0
`adapter`	100000	349.2	38.4	412

机器人求解	ns/op
`matrix3Inverse`	64,417
`chain4Forward`	270.6

8 数值正确性与标准化

类别	条件	结果
`denseSparse`	n=10, gp/sandwich	0.00e+00
`denseSparse`	n=4, gp/sandwich	0.00e+00
`denseSparse`	n=8, gp/sandwich	0.00e+00
`order`	n=8	0.00e+00
`order`	n=10	0.00e+00
`order`	n=12	0.00e+00
`cond`	eps=1e-03	3.45e-14
`cond`	eps=1e-08	1.93e-09
`cond`	eps=1e-12	7.88e+01
`sum`	double vs BigDecimal	0.00e+00
`joml`	float vs double	5.03e-05

导出的 ga-benchmark JSON 含 397 个逻辑 benchmark（794 条 mean/stddev 聚合）、 20 类算子，覆盖 Euclidean/Homogeneous/Conformal 的 D=3/4 与 grade 维度； `run-gabm.sh` 已完成生成、字段校验与上游 `gabm.py` 驱动。

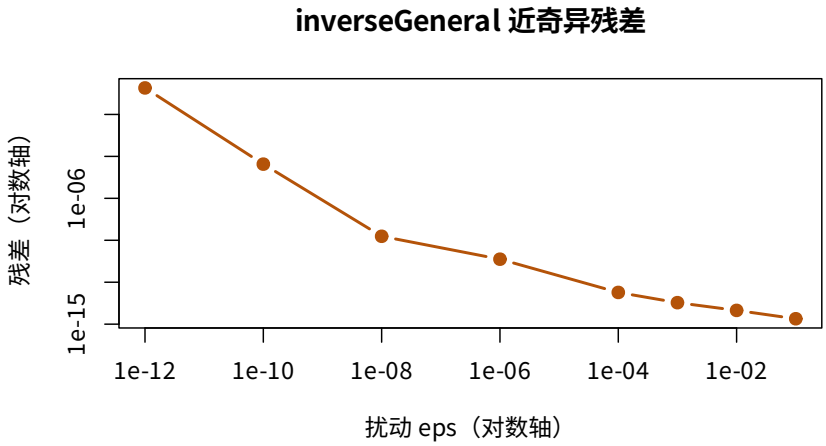


Figure 8: 近奇异元素的逆残差。

ga-benchmark 指标	值
benchmarks	397
aggregates	794
operations	20
models	1 2 3
dimensions	3 4
time_unit	ms

9 结论与路线

1. **稠密 gp 不要继续微优化**：输出驱动与 FMA 变体实测更慢 （n=10 为 1.47–1.50 ms，同 harness 基线 0.93 ms）；现有行优先符号表循环最接近 硬件上限。真正的结构收益来自稀疏支撑（Cl(10) 少非零时数百倍）与 子掩码缩并（n=10 约 8×）。
2. **稀疏路径按中间密度设计**：gp 可扩展，sandwich 在高密度中间结果 下退化；下一步应实现三重积的直接展开与不平衡树剪枝，而不是继续调阈值。
3. **批量受内存带宽限制**：复用输出把 1M 点从约 51.2 降到 20.0 ns/点、分配降为 0；4 线程达到平台，更多线程无收益。
4. **端到端差距来自算法层**：缓存 `Matrix4f` 比通用 GA 快一个 数量级；固定公式应生成/展开专用内核（Gaigen/Gaalop/GATL 的共同结论）。
5. **正确性**：稠密/稀疏逐位一致，子掩码与暴力参考一致； `inverseGeneral` 先做行列均衡化、两次迭代精化和后向误差检查， 近奇异结果默认拒绝，`inverseGeneralResult` 保留残差/条件数。

10 参考资料

- GA-Benchmark：`https://github.com/ga-developers/ga-benchmark`；本报告按其 Euclidean/Homogeneous/Conformal 算子集导出 JSON，沿用其 Model/D/Grade 维度。
- Gaigen 2、Gaalop、GATL：分别代表代码生成、符号展开与模板元编程三条 固定公式优化路线；端到端差距与它们的公开结果量级一致。
- JOML：`https://github.com/JOML-CI/JOML`，float 四元数与列主序矩阵对照。
- Macdonald, Linear and Geometric Algebra、Vector and Geometric Calculus：转子约定、场微分与面积加权采样。

· Prodanov, Algorithmic computation of multivector inverses and characteristic polynomials in non-degenerate Clifford algebras, arXiv:2308.02291：FVS 求逆路线。

· Higham, Iterative Refinement Implies Numerical Stability for Gaussian Elimination, Math. Comp. 35 (1980)：迭代精化与后向误差。

· LAPACK Users' Guide, How to Measure Errors：条件数与 RCOND。

· Breuils et al., Garamon: A Geometric Algebra Library Generator, Adv. Appl. Clifford Algebras (2019)：展开/生成代码的性能上限。

· Java Vector API: Benchmarking and Performance Analysis, ACM SIGPLAN (2023)：Vector API 的布局与收益边界。

11 复现与限制

```
cd lib/versor-java
./bench-all.sh                  # full test matrix
cd report && Rscript report2.R  # rebuild this report
```

限制：单机、非隔离、单线程为主；多核为 6P+8E/20 逻辑核；JOML 为 float； Vector API 仅作可选原型（incubator， Minecraft 启动参数不可控），生产路径仍用 标量；ga-benchmark 仅覆盖 Euclidean D=3/4、Homogeneous/Conformal D=3， 且上游尚未注册 `versor-java`，`run-gabm.sh` 在临时副本中补登记。